

IHK Methoden implementieren – Master Sheet

High-Score

Deutsch

فارسی

Ziel: In der AP2-Prüfung jede Methode schnell erkennen und im IHK-Stil korrekt implementieren.

هدف: در امتحان AP2 هر متد را سریع تشخیص بدهی و به سبک IHK درست بنویسی. این برگه «ماستر» الگوهای اصلی و ترفندها را جمع کرده است.

Allgemeine Checkliste / چک لیست کلی

- **Signatur lesen:** Rückgabetyt, Parameter, Datentypen.
- **Wort-Trigger:** suche, finde, zaehle, berechne, erstelle, sort, get.
- Struktur meist: **Initialisierung** → **Schleife** → **Bedingung** → **Aktion** → **Rückgabe**.
- Immer Datentyp passend zum Rückgabetyt / Parametern wählen.
- **امضای متد را دقیق بخوان:** نوع خروجی، پارامترها و نوع داده‌ها.
- **کلمات کلیدی سؤال:** مثل `suche` / `finde` (جستجو)، `zaehle` (شمارش)، `berechne` (محاسبه)، `erstelle` (لیست بساز)، `sort` (مرتب‌سازی)، `get` (برگرداندن یک مقدار).
- تقریباً همه‌جا الگو این است: **مقداردهی اولیه** → **حلقه** → **شرط** → **عمل** → **return**.
- همیشه نوع متغیرها را مطابق امضای متد انتخاب کن تا خطا نگیری.

1. Such-Methoden – boolean / Index / Objekt

Typische Namen: `getAppointment`, `enthaelt`, `sucheTopSeller`, `findeKunde`.

متدهای جستجو: معمولاً اسمشان با `suche` یا `finde` یا `get` ... شروع می‌شود و یک عنصر را در آرایه یا لیست پیدا می‌کند.

Pseudocode-Muster (IHK-Stil, boolean)

```
Funktion enthaelt(A: Array<T>, gesucht: T) → boolean
    n = länge(A)
    für i = 0 bis n - 1
        wenn A[i] = gesucht dann
            return true
        end wenn
    end für
    return false
end Funktion
```

Java-Grundform

```
public static <T> boolean enthaelt(T[] feld, T gesucht) {
    for (int i = 0; i < feld.length; i++) {
        if (feld[i].equals(gesucht)) {
            return true;
        }
    }
    return false;
}
```

Prüfungs-Tricks (Suche):

- Rückgabebetyp boolean → meist **sofortiges return** in der Schleife.
- Bei Index-Rückgabe: return i statt true, und am Ende -1.
- Bei Objekt-Rückgabe: return A[i], am Ende null.
- اگر خروجی boolean است، در لحظه پیدا شدن جواب بلافاصله **return** کن (نیازی به ادامه حلقه نیست).
- اگر خروجی Index است، به جای true عدد i را برگردان و در انتها اگر چیزی پیدا نشد -1 بده.
- اگر خروجی خود آبجکت است، در محل یافتن، A[i] را برگردان و در انتها null.

Beispiel AP2: getAppointment(month, day, id, slot): boolean – Arzt-Terminplan prüfen.

2. Zählen mit Bedingung – zaehleMitBedingung

Schlüsselwörter: wie viele, Anzahl, count, zaehle.

هر جا سؤال پرسید «چند تا...؟» یا «Anzahl...»، تقریباً حتماً از الگوی شمارش با شرط استفاده می‌شود.

Pseudocode-Muster

```
Funktion zaehleMitBedingung(A: Array<T>) → Integer
    n = länge(A)
    anzahl = 0

    für i = 0 bis n - 1
        wenn <Bedingung für A[i]> dann
            anzahl = anzahl + 1
        end wenn
    end für

    return anzahl
end Funktion
```

Beispiel (Minimum IHK-Stil)

z. B. Anzahl Kunden mit Umsatz $\geq$ Limit:

```

Funktion zaehleKundenMitLimit(K: Array<Kunde>, limit: Real) → Integer
    n = länge(K)
    anzahl = 0
    für i = 0 bis n - 1
        wenn K[i].umsatz ≥ limit dann
            anzahl = anzahl + 1
        end wenn
    end für
    return anzahl
end Funktion

```

Prüfungs-Tricks (Zählen):

- Zähler vor der Schleife mit 0 initialisieren.
- In der Schleife **nur bei erfüllter Bedingung** erhöhen.
- Nie in der Schleife zurückgeben, immer return anzahl nach der Schleife.
- متغیر شمارنده را همیشه قبل از حلقه با صفر مقداردهی کن.
- فقط وقتی شرط برقرار است anzahl را یکی زیاد کن.
- در این الگو هرگز در وسط حلقه return نکن، همیشه بعد از حلقه return anzahl می‌آید.

3. Minimum / Maximum / Durchschnitt

3.1 Minimum

```

Funktion findeMinimum(W: Array<Integer>) → Integer
    min = W[0]
    für i = 1 bis länge(W) - 1
        wenn W[i] < min dann
            min = W[i]
        end wenn
    end für
    return min
end Funktion

```

3.2 Maximum

```

Funktion findeMaximum(W: Array<Integer>) → Integer
    max = W[0]
    für i = 1 bis länge(W) - 1
        wenn W[i] > max dann
            max = W[i]
        end wenn
    end für
    return max
end Funktion

```

```
end für
return max
end Funktion
```

3.3 Durchschnitt (arithm. Mittel)

```
Funktion berechneDurchschnitt(A: Array<Integer>) → Real
  wenn länge(A) = 0 dann
    return 0.0
  end wenn

  summe = 0
  für i = 0 bis länge(A) - 1
    summe = summe + A[i]
  end für

  return summe / länge(A)
end Funktion
```

Tricks Min/Max/Durchschnitt:

- Startwert Min/Max immer mit A[0], nicht mit 0.
- Durchschnitt nie in der Schleife ausrechnen – erst **summe**, dann teilen.
- Bei Division auf Real achten (Kommazahl nötig).
- برای Min/Max همیشه مقدار اولیه را از عنصر اول آرایه بگیر (A[0]) نه از صفر.
- میانگین را داخل حلقه حساب نکن، فقط جمع را بساز و در آخر تقسیم کن.
- اگر خروجی اعشاری است، نوع را Real / double بگیر تا جواب صحیح شود.

4. Liste aufbauen / filtern – erstelle_liste

Namen: `erstelle_liste`, `filtereMitarbeiter`, `baueBestellListe`, Rückgabe ist meist `Liste<T>`.

هر جا خروجی «لیست» باشد (مثل `Liste<Mitarbeiter>`), معمولاً باید یک لیست جدید بسازی و فقط عناصری که شرط را پاس می‌کنند به آن اضافه کنی.

Pseudocode-Muster

```
Funktion erstelle_liste(stundensatz: Real) → Liste<Mitarbeiter>
  liste = neueListe()
  n = länge(mitarbeiter)

  für i = 0 bis n - 1
```

```

        lohn = mitarbeiter[i].stunden * stundensatz
        wenn lohn ≥ mindestLohn dann
            liste.hinzufuegen(mitarbeiter[i])
        end wenn
    end für

    return liste
end Funktion

```

neueListe() bedeutet: eine **leere** Liste erzeugen, in Pseudocode ohne genaue Implementierung.

تابع `neueListe()` یعنی «یک لیست خالی جدید بساز». در شبه‌کد لازم نیست نوع دقیق پیاده‌سازی (`ArrayList` و غیره) را بنویسی.

Tricks Listenaufbau:

- Immer: `liste = neueListe()` vor der Schleife.
- Innerhalb der Schleife nur bei erfüllter Bedingung `liste.hinzufuegen(...)`.
- Nie `return` in der Schleife, immer nach der Schleife.
- قبل از حلقه حتماً `liste = neueListe()` بنویس.
- فقط وقتی شرط برقرار است عنصر را به لیست اضافه کن.
- در این الگو، مثل شمارش، فقط یک `return` در انتها داریم: `return liste`.

5. Vergleich / Sortier-Helfer – vergleiche..., sort(...)

In vielen Prüfungen kommen Methoden wie `vergleicheMitarbeiter` oder `sort(räume: Array von Raum)` vor. Sie dienen oft nur als Hilfsmethoden für ein Sortierverfahren.

در خیلی از سال‌ها متدهای `vergleiche...` یا `sort(...)` آمده که معمولاً برای مرتب‌سازی استفاده می‌شوند.

Vergleich zweier Objekte (IHK-Stil)

```

Funktion vergleicheMitarbeiter(m1: Mitarbeiter, m2: Mitarbeiter) → Integer
    wenn m1.gehalt < m2.gehalt dann
        return -1
    end wenn
    wenn m1.gehalt > m2.gehalt dann
        return 1
    end wenn
    return 0
end Funktion

```

Rückgabe-Bedeutung: $-1 = m1 < m2$, $1 = m1 > m2$, $0 = \text{gleich}$.

تفسیر خروجی: 1- یعنی m1 از m2 کوچکتر است، 1 یعنی بزرگتر و 0 یعنی مساوی.

Sortier-Schema (Bubble-ähnlich)

```
Prozedur sort(raeume: Array von Raum)
  n = länge(raeume)
  für i = 0 bis n - 2
    für j = 0 bis n - 2 - i
      wenn vergleicheRaum(raeume[j], raeume[j+1]) > 0 dann
        // tausche
        temp = raeume[j]
        raeume[j] = raeume[j+1]
        raeume[j+1] = temp
      end wenn
    end für
  end für
end Prozedur
```

Tricks Vergleich/Sort:

- Vergleichs-Methode ist sehr kurz, nur if/else + return -1/0/1.
- Beim Sortieren nie neue Liste zurückgeben (wenn nicht verlangt). Meist wird Array **in-place** sortiert.
- متد مقایسه همیشه خیلی کوتاه است: چند تا if و در نهایت 1- یا 0 یا 1 برمی‌گرداند.
- در مرتب‌سازی معمولاً همان آرایه ورودی را جابه‌جا می‌کنیم و چیز جدیدی برنمی‌گردانیم مگر این‌که صریحاً در سؤال خواسته شود.

6. OOP-Muster – Strategie / Delegation

(getVersandkosten())

Typischer Fall wie im Warenkorb-Beispiel: Klasse Warenkorb hat ein Attribut versand: Versand (Interface). Die Methode getVersandkosten() delegiert die Berechnung an das Strategie-Objekt.

یک سناریوی کلاسیک: کلاس Warenkorb یک فیلد versand از نوع اینترفیس Versand دارد. متد getVersandkosten() خودش حساب نمی‌کند، فقط کار را به آبجکت versand می‌سپارد.

Pseudocode-Implementierung

```
Klasse Warenkorb
  // Attribute (vereinfacht)
  artikel : Artikel[]
  anzahl  : Integer[]
  versand : Versand
```

```

Methode getVersandkosten() : double
    return versand.berechnen(this)
end Methode
Ende Klasse

```

Tricks OOP/Delegation:

- Wenn ein Interface mit Methode berechnen(Warenkorb): double gegeben ist, dann ruft die Klasse meist nur versand.berechnen(this) auf.
- **this** steht für die aktuelle Instanz (den aktuellen Warenkorb).
- In Pseudocode reicht oft genau eine Zeile im Methoden-Körper.
 - اگر یک Interface مثل Versand با متد berechnen(Warenkorb) داده شده، تقریباً همیشه در کلاس Warenkorb باید فقط بنویسی: return versand.berechnen(this).
 - this یعنی همین شی فعلی (این Warenkorb).
 - بدون اضافه کاری: در چنین سؤالاتی معمولاً کل بدنه متد فقط یک خط است.

7. Umrechnung / Schleifenarithmetik – z. B.

Umrechnung(dezimalzahl: int)

Aufgaben wie Dezimal → Binär, Sekunden → Stunden/Minuten, Währungskurs: immer gleiche Idee: **solange teilen oder mod, Ergebnis merken, am Ende formatieren.**

مسائل تبدیل عدد (دسیمال به باینری، ثانیه به ساعت/دقیقه و غیره) همه یک ایده دارند: با تقسیم و باقیمانده کار می‌کنی و نتیجه را در یک رشته یا آرایه ذخیره می‌کنی.

Pseudocode Bsp. (Dezimal → Binär als String)

```

Funktion Umrechnung(dezimalzahl: Integer) → String
    wenn dezimalzahl = 0 dann
        return "0"
    end wenn

    ergebnis = ""           // leerer String
    zahl = dezimalzahl

    solange zahl > 0
        rest = zahl mod 2
        ergebnis = "" + rest + ergebnis    // vorne anhängen
        zahl = zahl / 2
    end solange

```

```
return ergebnis
end Funktion
```

Tricks Umrechnung:

- Immer mit Hilfsvariable `zahl` arbeiten, nicht direkt mit Parameter.
- Schleife mit `solange zahl > 0`, in jedem Schritt: `rest = mod, zahl = zahl / Basis`.
- Bei Ausgabe von Zahlenfolgen oft „von hinten nach vorne“ anhängen.
- برای تبدیل‌ها از یک متغیر کمکی مثل `zahl` استفاده کن و خود پارامتر را دست‌کاری نکن.
- الگوی کلی: تا وقتی `zahl > 0` است → باقیمانده بگیر، خود عدد را تقسیم کن و نتیجه را در رشته/آرایه ذخیره کن.
- اگر ترتیب برعکس است (مثل باینری)، رقم‌های جدید را در ابتدای رشته اضافه کن.

8. Zusammenfassung – Muster schnell erkennen

- **boolean + „gibt es ...?“** → Muster 1 (Suche).
- **„Wie viele ...?“** → Muster 2 (Zählen).
- **„kleinster / größter / Durchschnitt“** → Muster 3.
- **„Liste von ... zurückgeben“** → Muster 4 (Liste bauen).
- **„Sortieren“ oder „vergleiche...“** → Muster 5.
- **Interface + this** → Muster 6 (Delegation/Strategie).
- **„umrechnen“ / „Konvertieren“** → Muster 7.
- **boolean و سؤال «آیا ممکن است؟»** → الگوی ۱ (جستجو).
- **«چند تا؟»** → الگوی ۲ (شمارش).
- **کمترین، بیشترین، میانگین** → الگوی ۳.
- **خروجی لیست** → الگوی ۴ (ساخت لیست جدید).
- **مرتب‌سازی یا مقایسه** → الگوی ۵.
- **اینترفیس + this** → الگوی ۶ (Delegation / Strategy).
- **تبدیل واحد یا سیستم عددی** → الگوی ۷.